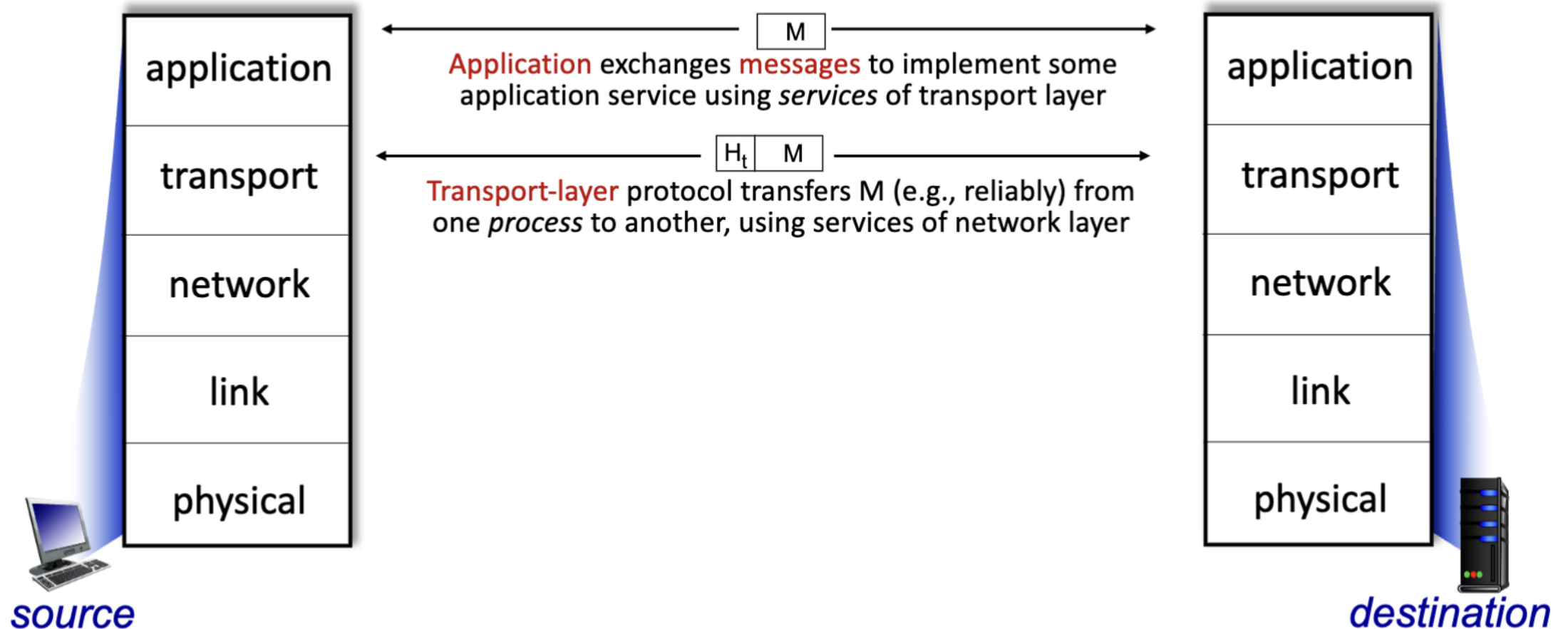


Review

Network Layers and Services



Review

Client-Server Paradigm

server:

- always-on host
- permanent IP address
- often in data centers, for scaling

clients:

- contact, communicate with server
- may be intermittently connected
- may have dynamic IP addresses
- do *not* communicate directly with each other
- examples: HTTP, IMAP, FTP

Application layer: overview

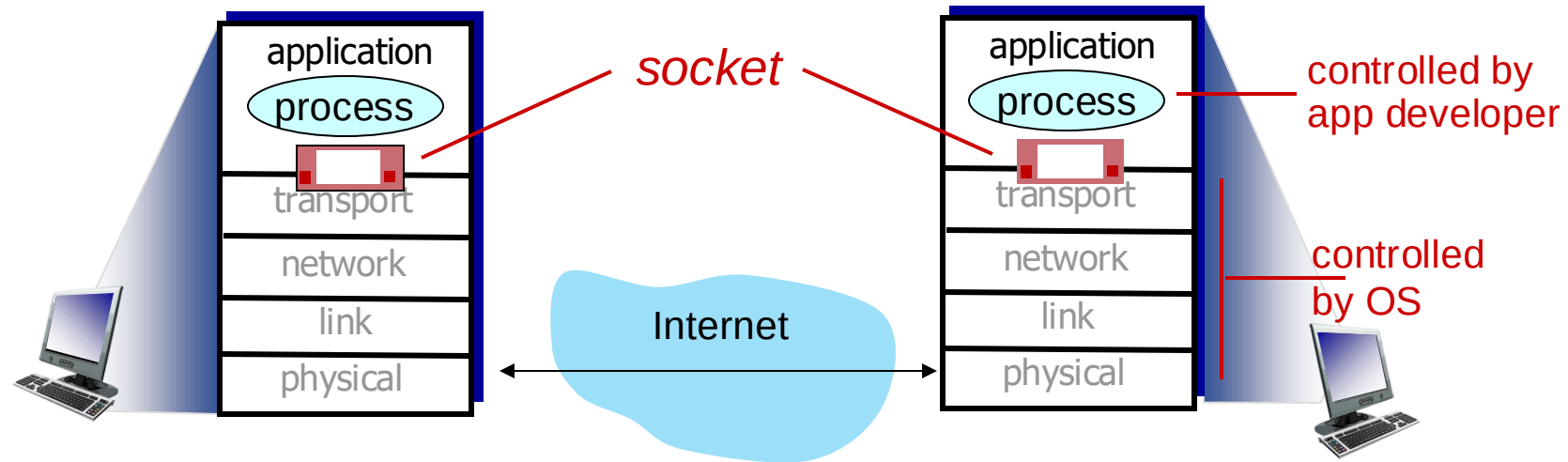
- Principles of network applications
- Web and HTTP
- video streaming and content distribution networks
- socket programming with UDP and TCP
- The Domain Name System (DNS)
- P2P applications

Lecture goals

- Define role of TCP and UDP sockets in network communication
- Learn how to build client/server applications that communicate using sockets
- Program a socket-based application in Python

Socket programming

Socket: door between application process and end-end-transport protocol



Socket programming

Two socket types for two transport services:

- *UDP*: unreliable datagram
- *TCP*: reliable, byte stream-oriented

Socket programming with UDP

UDP: no “connection” between client and server:

- no handshaking before sending data
- sender explicitly attaches IP destination address and port # to each packet
- receiver extracts sender IP address and port# from received packet

UDP: transmitted data may be lost or received out-of-order

Application viewpoint:

- UDP provides *unreliable* transfer of groups of bytes (“datagrams”) between client and server processes

Client/server socket interaction: UDP



server (running on serverIP)

create socket, port= x:
`serverSocket =
socket(AF_INET,SOCK_DGRAM)`

read datagram from
`serverSocket`

write reply to
`serverSocket`
specifying
client address,
port number

client



create socket:
`clientSocket =
socket(AF_INET,SOCK_DGRAM)`

Create datagram with serverIP address
And port=x; send datagram via
`clientSocket`

read datagram from
`clientSocket`

close
`clientSocket`

Example Application

1. client reads a line of characters (data) from its keyboard and sends data to server
2. server receives the data and converts characters to uppercase
3. server sends modified data to client
4. client receives modified data and displays line on its screen

Example app: UDP client

Python UDPClient

include Python's socket library	→	from socket import *
		serverName = 'hostname'
		serverPort = 12000
create UDP socket	→	clientSocket = socket(AF_INET, SOCK_DGRAM)
get user keyboard input	→	message = input('Input lowercase sentence:')
attach server name, port to message; send into socket	→	clientSocket.sendto(message.encode(), (serverName, serverPort))
read reply data (bytes) from socket	→	modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
print out received string and close socket	→	print(modifiedMessage.decode()) clientSocket.close()

Example app: UDP server

Python UDPServer

```
from socket import *
serverPort = 12000
create UDP socket → serverSocket = socket(AF_INET, SOCK_DGRAM)
bind socket to local port number 12000 → serverSocket.bind(('', serverPort))
print('The server is ready to receive')
loop forever → while True:
    Read from UDP socket into message, getting → message, clientAddress = serverSocket.recvfrom(2048)
    client's address (client IP and port)      modifiedMessage = message.decode().upper()
    send upper case string back to this client → serverSocket.sendto(modifiedMessage.encode(),
                                                                    clientAddress)
```

Socket programming with TCP

Client must contact server

- server process must first be running
- server must have created socket (door) that welcomes client's contact

Client contacts server by:

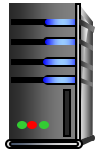
- Creating TCP socket, specifying IP address, port number of server process
- *when client creates socket*: client TCP establishes connection to server TCP

- when contacted by client, *server TCP creates new socket* for server process to communicate with that particular client
 - allows server to talk with multiple clients
 - client source port # and IP address used to distinguish clients (more in Chap 3)

Application viewpoint

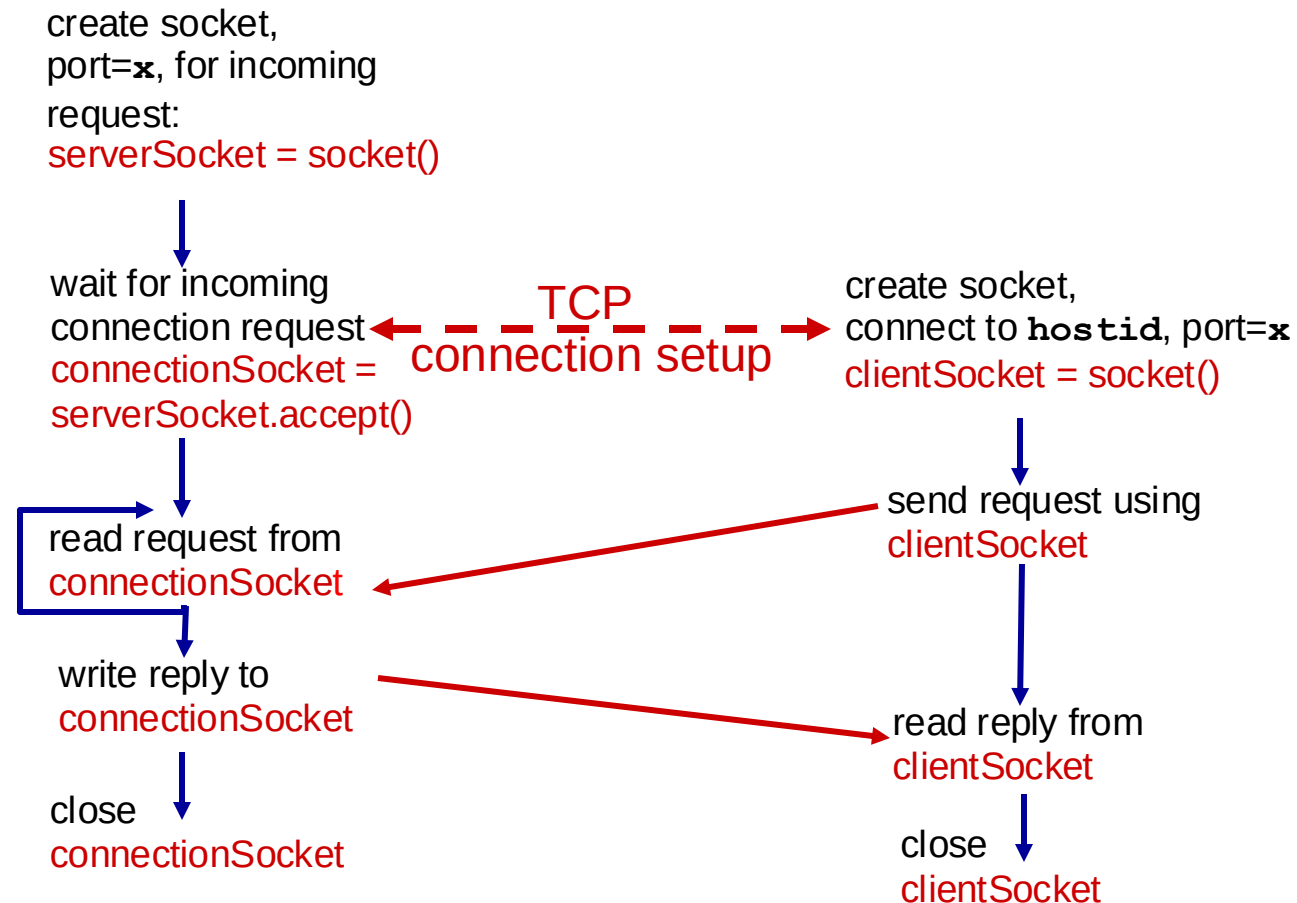
TCP provides reliable, in-order byte-stream transfer ("pipe") between client and server processes

Client/server socket interaction: TCP



server (running on `hostid`)

client



Example app: TCP client

Python TCPClient

create TCP socket for server,
remote port 12000

```
from socket import *
serverName = 'servername'
serverPort = 12000
clientSocket = socket(AF_INET, SOCK_STREAM)
clientSocket.connect((serverName, serverPort))
sentence = input('Input lowercase sentence:')
clientSocket.send(sentence.encode())
modifiedSentence = clientSocket.recv(1024)
print ('From Server:', modifiedSentence.decode())
clientSocket.close()
```

No need to attach server name, port

Example app: TCP server

Python TCP Server

		<pre>from socket import *</pre>
		<pre>serverPort = 12000</pre>
create TCP welcoming socket	→	<pre>serverSocket = socket(AF_INET, SOCK_STREAM)</pre>
		<pre>serverSocket.bind(('', serverPort))</pre>
server begins listening for incoming TCP requests	→	<pre>serverSocket.listen(1)</pre>
		<pre>print('The server is ready to receive')</pre>
loop forever	→	<pre>while True:</pre>
server waits on accept() for incoming requests, new socket created on return	→	<pre> connectionSocket, addr = serverSocket.accept()</pre>
		<pre> sentence = connectionSocket.recv(1024).decode()</pre>
read bytes from socket (but not address as in UDP)	→	<pre> capitalizedSentence = sentence.upper()</pre>
		<pre> connectionSocket.send(capitalizedSentence.encode())</pre>
close connection to this client (but <i>not</i> welcoming socket)	→	<pre> connectionSocket.close()</pre>

Note: this code update (2023) to Python 3